A High-Fidelity Emulation Mechanism Reproducing Network-Observable Behavior of ICS Devices for Cyber Resilience Assessment

KAZUMA KASAHARA[1,a)] SATOSHI KAI[1,b)] TAKAO KONDO[1,c)]

Abstract: Assessing the resilience of an industrial control system (ICS) requires penetration testing, but testing on a live system endangers safety and availability, so an isolated reference system that combines real devices with emulators is used instead. However, existing ICS device emulators cannot reproduce the vendor-specific and site-specific customizations, whose specifications are often unavailable. We propose a high-fidelity emulation mechanism that reproduces the input and output behavior an ICS device exposes on the network. A static finite-state machine (FSM) fixes the packet structure, and a fine-tuned large language model (LLM) generates the vendor-specific and site-specific fields. Both are built only from captured traffic, so the mechanism needs no vendor specification, and supporting a second protocol requires a declarative description of it rather than new runtime code. Using our proposed mechanism, we emulate three real devices in a smart grid across two protocols: two intelligent electronic devices (IEDs) using IEC 61850 and a station computer using IEC 60870-5-104. We measure fidelity by replaying captured traffic against each emulator and comparing its responses with the real device's, field by field. Under cross-validation the emulators reproduce 93.7 to 98.9 % of the responses a client requests, every emitted response is structurally valid, and repeated runs are identical. Responses the device sends unprompted are reproduced at 1.6 and 2.0 %, which is the present limitation of the mechanism: their content depends on plant state it does not yet model, and modeling that state is the next step. An operator able to capture a device's packets can therefore emulate its request-response behavior with high fidelity on commodity hardware.

Keywords: ICS security, penetration testing, protocol emulation, IEC 61850, IEC 60870-5-104

1. Introduction

An industrial control system (ICS) is the hardware and software that monitors and controls a physical process. In an electrical substation it is the intelligent electronic devices (IEDs) that measure the plant and operate the switchgear, and the station computer that supervises them. Unlike an information system, an ICS is engineered for safety and availability, runs for decades, and is rarely patched.

The electricity grid is critical infrastructure on which every other sector depends, so an outage propagates at once and widely, and the grid is correspondingly attractive to well-resourced attackers [24]. The protocols its devices speak, including the Manufacturing Message Specification (MMS) of IEC 61850 [12] and IEC 60870-5-104 [14], were standardized without authentication, and the security extensions that exist are rarely deployed. Security guidance accordingly treats network segmentation, rather than authentication at the device, as the control that keeps such a device out of reach [13], [24], and that control is not always sufficient. Industroyer, the malware behind the 2016 Kyiv blackout, carried payloads that issue control commands over IEC 61850 and IEC 60870-5-104 among other protocols [5]. Its 2022 successor, built to operate substation protection relays over IEC 60870-5-104, was detected and removed before it ran [8]. The resilience of such a system is judged by how it behaves under attack, so its assessment requires penetration testing.

A live plant cannot be the target of that testing, because safety and availability take precedence over every security objective. The accepted answer is an isolated reference system: a testbed that reproduces the production environment closely enough that attacks can be executed and their effects measured [7], [16]. Building a substation reference system from real IEDs and station computers alone is bounded by procurement cost, rack space, and the fact that vendors do not sell a spare of every installed unit, so the testbed literature converges on a hybrid form in which a few real devices operate beside many emulated ones [7], [9]. An emulated device fills its place in such a system only if its network behavior cannot be distinguished from the behavior of the device it stands in for, because the tester's tooling reads the wire and nothing else.

Existing emulators do not meet this condition for installed ICS devices. An installed device combines a standard protocol with customization that differs by vendor and by site: engineering configuration files, private extensions, and plant-specific point maps. That customization is usually undocumented for the operator, so

[1] Keio University, Japan
[a)] k4zuma-k4sahara0277@keio.jp
[b)] skai@sfc.keio.ac.jp
[c)] latte@adst.keio.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

an emulator driven by the public protocol specification reproduces the standard behavior and misses the site-specific behavior. We instead learn the behavior from captured traffic: construction consumes packet captures and operator-supplied configuration, and assumes no vendor documentation.

This paper continues our earlier report on the same emulation mechanism [17]. That report introduced the division of labor the mechanism rests on. A static finite-state machine (FSM), built offline from captured traffic, owns the structure of every packet, and each field of a packet is classified once at construction time into one of three kinds: a recorded constant, a value computed from the request, or a value produced by a fine-tuned large language model (LLM). The evaluation there covered one protocol, IEC 61850, with two IEDs from one site, and it reported the accuracy of the generated values together with the response latency measured against the real device.

The same report named the two limits that this paper removes. First, classification was keyed on the name the packet dissector gives a field, so one name that carries two unlike populations of values was treated as a single kind, and values that no model can predict were sent to the LLM. Second, state merging during construction used the sequence of message types alone, which collapsed whole sessions into a single state on our captures and left the connection lifecycle unrepresented. This paper rebuilds both parts. It also changes what is measured. The earlier report measured the accuracy of the generated values in isolation, whereas this paper measures the fidelity of the complete response against the real device together with the reproducibility of that response across repeated runs, and it adds a third device, a station computer speaking IEC 60870-5-104, so that the claim of protocol independence is tested rather than asserted.

This paper asks four research questions. Each is answered by one part of the evaluation, and Section 5 states which subsection answers which question.

- **RQ1 (fidelity).** Can an emulator built only from captured traffic reproduce the responses the real device gives to the same request sequence, byte for byte outside the fields that are declared exempt in advance?
- **RQ2 (reproducibility).** Is the same request sequence answered identically across runs, so that an assessment result can be replicated?
- **RQ3 (versatility).** Does supporting a second protocol require only captured traffic and a declarative description, with no new runtime code?
- **RQ4 (cost).** Do construction and deployment run on commodity hardware at a cost an operator can afford per device?

The contributions of this paper are as follows.

(1) A field classification procedure that decides how each protocol field is reproduced from the population of values the field takes, rather than from the name the dissector gives it.

(2) An FSM construction procedure that preserves the connection lifecycle under state merging, pairs requests with responses by correlation identifier, and confirms its merge constraints against the traces instead of taking them per device from an operator.

(3) A case study on three real devices in a smart grid speaking two protocols, two IEDs using IEC 61850 and a station computer using IEC 60870-5-104, built with the same code base and differing only in captured traffic and declarative description.

(4) A fidelity and reproducibility measurement under cross-validation: each capture is replayed against an emulator built only from the other captures, so no result rests on a response it had already seen.

2. Related Work

2.1 Simulators and testbeds for ICS security

Surveys of ICS and smart grid security testbeds report a consistent architecture: physical process simulation, control logic emulation, and standard protocol stacks composed into a laboratory replica of a plant [6], [7]. The instances differ mainly in how much of that composition is real. At one end a testbed is built from operational hardware, as in the SWaT water treatment plant [20] or the reference testbed built to measure the cost of security controls on real control equipment [3]; the smart grid equivalent is a substation bay populated with real IEDs and a real station computer [6]. Fidelity is then not in question, but cost, footprint, and a fixed device inventory are the limits our work addresses. At the other end, MiniCPS composes network emulation with simulated controllers speaking standard industrial protocols [1]. Between them, practical guidance treats real, emulated, and simulated components as interchangeable building blocks chosen by which properties an experiment must preserve [9]. Our mechanism raises the fidelity available in that middle range. It supplies an emulated component that answers the tester's traffic with the same bytes the installed unit would return, so a device can move from the real side of a testbed to the emulated side without changing what the tester sees.

Closest in aim is the automated generation of a smart grid cyber range from IEC 61850 engineering models, which instantiates a substation network, including its IEDs, from the standard configuration description [23]. The difference is which input is trusted. A generated range is faithful to the model, so it reproduces what the engineering files specify and nothing they omit, while we treat captured traffic as the specification and so reproduce vendor extensions and any divergence between the configuration and the running device. The two are complementary: models supply breadth across a substation, captures supply depth at the device under attack. The contribution we claim is accordingly specific: not a testbed architecture, but higher fidelity for the devices inside one, at the level a penetration test observes.

2.2 Honeypots and LLM-based protocol emulation

ICS honeypots pursue a related but distinct goal. HoneyPLC emulates vendor-specific controller profiles so that a scan cannot separate it from a real controller [19]. Its purpose is to attract an attacker, so plausibility suffices where we require correspondence with one installed device. Two recent systems apply LLMs to the same problem, and both also target deception. LLMPot fine-tunes a byte-level model on request and response pairs and generates each response end to end, reporting 99.30 % valid responses on Modbus and 160 ms per response on a desktop GPU [25]. Be-

© 2026 Information Processing Society of Japan

cause the model emits the entire packet, there is nothing to constrain the output structurally: framing, length fields, and nested encodings all sit inside the model output. The published results cover register-based protocols, not layered ASN.1 encodings whose length cascades collapse under single-byte errors. The combination of a finite-state machine with retrieval-augmented generation is the closest algorithmic prior work [4]. It builds a Mealy machine from traces and extends the machine at runtime when an unknown request arrives. Runtime extension has two costs that our setting cannot absorb. Its median response time rises from 5 to 30 s once its output is checked against the protocol specification, where retrieval alone takes 30 to 40 ms, and two runs of the same attack can diverge once the machine has grown differently. A deception honeypot tolerates both. An emulator used for assessment tolerates neither, because when a measurement cannot be reproduced, the behavior of the tool under test cannot be separated from variation in the emulator. Our state machine is therefore built offline and is immutable at runtime.

2.3 Protocol behavior inference from traces

The system above that combines a finite-state machine with retrieval-augmented generation [4] infers its state machine from traces, and that lineage is where our construction differs from it. ReverX infers both message formats and protocol state from network traces, and is the method that work follows [2], [4]. The merge step itself we take from evidence-driven state merging, which makes inference practical on sparse data by merging the states of a prefix tree whose observed evidence agrees [18]; that prior system leaves the merge relation abstract and never fixes a concrete algorithm, so this choice is ours rather than inherited. Both methods decide a merge from the history of message types alone. On our captures of the complex protocols used in the case study, that evidence is insufficient: sessions differ in payload values, not in message order, and history-only merging collapses every session into one state. The construction procedure of Section 3.3 therefore conditions merging on payload content, and derives the conditioning fields from the data instead of taking them from an operator. We do not infer message formats, as ReverX must, because standard dissectors already decode the protocols we target; inference is confined to state topology and field behavior.

3. System Design

We define the network-observable behavior of a device as the mapping from the byte sequence the device receives to the byte sequence it sends, including the traffic the device originates on its own initiative. This mapping is the object the mechanism reproduces. Anything not on the wire, such as firmware internals or physical effects, is out of scope by construction and remains the responsibility of the real devices in the reference system. Construction consumes captured traffic and operator-supplied configuration only; no vendor specification is assumed.

3.1 Inherited architecture

The earlier report [17] established the division of labor that this paper builds on, and we restate it briefly so that the boundary between inherited and new parts is explicit. Construction is the four-stage offline pipeline of Figure 1: capture selection, FSM construction, field evaluation, and LLM fine-tuning. FSM construction produces a deterministic Mealy machine whose input alphabet is a set of abstract message types; a declarative per-protocol description maps dissected packets onto that alphabet, so the alphabet and its dispatch rules are data rather than code. Field evaluation classifies every field once, at construction time, and the classification decides how it is rendered: a recorded constant is replayed, a computable field is recomputed from the request, a field that follows the device state is rendered from it, and only the residual goes to an LLM fine-tuned for each device to be emulated: a 0.5-billion-parameter instruction-tuned model, Qwen2.5-0.5B-Instruct [22], adapted with low-rank adaptation (LoRA) [10]. At runtime, the state machine fixes which response follows which request and the structure of every packet, while the LLM path writes value bytes into a recorded template at a recorded byte position. The LLM therefore never produces a header, a type tag, or a length field, so a wrong prediction corrupts one value rather than the packet. The LLM path is also fail-soft: on a timeout, an error, or a value of the wrong width, the field falls back to its recorded template value, so a failure of the LLM degrades one value to a previously observed one instead of malforming the response. Transport concerns live outside the state machine, which is immutable once built. Immutability makes RQ2 a design property rather than an empirical accident: given the same protocol state and the same request, the only component that can vary is the clock a timestamp rule reads.

Figure 2 shows what a request meets at runtime. Inherited from the earlier report are this division of labor, the protocol description contract, and the runtime skeleton; new in this paper are the two mechanisms below. Message-type identification now reads its field offsets from the declarative protocol description rather than from protocol-specific code. The offsets were the last protocol knowledge inside the identification step, so moving them into the description lets a second protocol arrive as data rather than as code. The field renderer gained a fourth path, which renders a field from the device state earlier commands established. Such a field is deterministic once the command history is known, so rendering it from tracked state takes it off the LLM path and replaces a learned approximation with an exact computation. The figure also shows where RQ3 is settled at runtime: the parser's transport stage is the only component written per protocol, so a second protocol reaches the runtime through its description and its artifact, not through new runtime code.

3.2 Field classification from value populations

The classifier of the earlier report decided how a field is reproduced from the field's dissector name, the identifier a packet dissector assigns to every field it decodes. The dissector name is the wrong classification key in two ways. First, one name can cover several unlike value populations: on IEC 61850 report payloads the dissector name `mms.data_bit_string` labels, at different positions of the same packet, both a switch position and a quality descriptor. The classifier must then give one verdict to values that behave differently, and precision is lost in both directions: values

© 2026 Information Processing Society of Japan

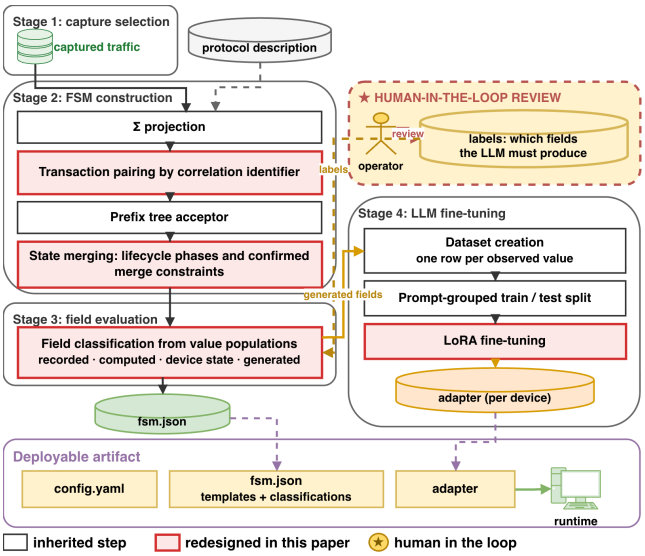


Fig. 1 Construction of one device artifact through the four offline stages. The operator reviews which fields the LLM must produce; those fields drive dataset creation, the prompt-grouped split, and per-device fine-tuning. Highlighted steps are the ones this paper redesigns.

that no model can predict are sent to the LLM path, and values that a rule could compute are learned instead of computed. Second, what a name carries is fixed by each device's engineering configuration, so a name-keyed verdict cannot be written down once and reused, and the earlier workflow relied on an operator to declare per device which fields a rule could compute. That declaration is the expensive step of onboarding a device: the operator must inspect the captures field by field, with knowledge of both the protocol and the site, and must repeat the whole inspection for every additional device.

The new classifier, the highlighted step of stage 3 in Figure 1, decides from the values themselves. It first separates occurrences of the same name into distinct populations, using a context the protocol description declares and the position of the occurrence in the packet, and, where the protocol pairs a request with its response, it attaches to each observed value the request that caused it. It then runs a fixed sequence of tests over every population, and the first test that matches decides the verdict. A population whose values equal a field of the causing request is computed by an echo rule. A population that advances monotonically, that renders the capture time, or that equals the length of the frame carrying it is computed by a counter, timestamp, or length rule. A population that is constant within a session, or whose values are opaque tokens with no learnable regularity, such as buffered-report entry identifiers, is replayed from the recorded template, because replay is exact when a population has no structure. A population determined by the device state earlier commands established, such as a switch position following the most recent control command, is rendered from tracked state. Only a population that fails every test reaches the LLM path. That residual is the site-specific variance which alone justifies a learned component. These verdicts are the four kinds the stage-3 box of Figure 1 lists, and an operator can override any of them through a per-device label file, the review step of the same figure.

This reclassification helps in both directions. More of each response is correct by construction, since a computed or replayed value is exact where a learned one is approximate. And the LLM's task becomes cleaner, since its training data no longer mixes predictable values into the residual it is meant to learn.

3.3 FSM construction

The construction of the earlier report merged states of the prefix tree acceptor using the sequence of message types alone, and on our captures it collapsed every session into a single state, leaving the connection lifecycle unrepresented. The redesign makes three changes, listed in dependency order. In Figure 1 they are the two highlighted steps of stage 2: the pairing step carries the first change, and the state-merging step carries the second and third.

First, transactions are paired by correlation identifier. Clients pipeline requests, so the packet that follows a request on the wire is often not its response. Every protocol we target carries a correlation identifier, and the protocol description declares which field it is; sessions are reordered into transaction order before any later construction step. Two later steps depend on this order. The classifier of Section 3.2 needs to know which request caused each response value, and pairing supplies that request. The third change below reasons over which message types follow which; in a pipelined session the wire order shows a request followed by the response of a different request, and pairing removes these false neighbors.

Second, connection lifecycles are segmented into phases. Transport setup, association, data transfer, and release are distinct phases of a connection, and the protocol description assigns each message type to one of them. Merging is forbidden across phases. Compaction can then no longer collapse a whole session into one state, because the lifecycle boundary survives every merge decision.

Third, merge constraints are confirmed from the traces rather than chosen by hand. The distinctions that matter on our captures lie in payload values rather than in message order, so merging must be conditioned on payload fields, and the construction of the earlier report required an operator to declare per device which field to condition on. The protocol description lists candidate fields once per protocol, taken from the fields whose value selects the service, the object, or the operation inside one message envelope, and the procedure decides which earn a constraint on the device at hand, where a constraint forbids merging two states whose observed values on the field never overlap. States are compared for merging by their outputs up to a lookahead of $k = 2$, the smallest that spans a complete transaction. For each candidate it evaluates the merges the field would forbid against two tests for bad merges: a merge is bad when it creates an output conflict deeper in the tree, or when it generalizes to transaction sequences that never occur in the paired traces. A candidate is accepted when at least 90 % of the merges it forbids are marked bad by at least one of the two tests. The decision is taken on the prefix tree, before any merge, so it does not depend on merge order.

4. Case Study in a Smart Grid

We applied the mechanism to the three devices of the smart grid testbed shown in Figure 3. They form one control path rather than three unrelated targets: a supervisory control and data ac-

© 2026 Information Processing Society of Japan

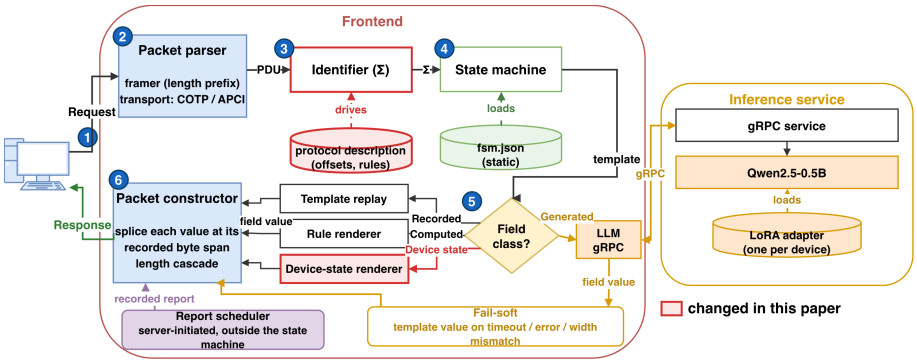


Fig. 2 Runtime path of one request. The packet parser's transport stage is the only per-protocol code; the identifier is driven by the declarative protocol description, and the state machine and its templates come from the artifact. Highlighted elements are the ones this paper changes.

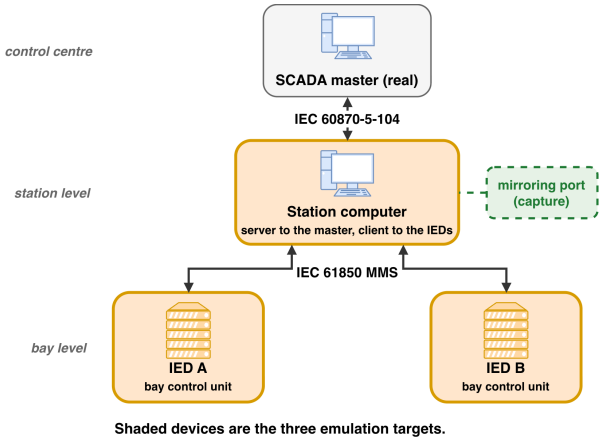


Fig. 3 The case study devices and the protocols between them.

quisition (SCADA) master supervises a station computer, and the station computer supervises two bay control unit IEDs. The station computer is therefore a server on one interface and a client on the other, which is why two protocols meet in a single system. They are of deliberately different character. IEC 61850 MMS is a session-oriented object-access protocol carried over a layered OSI stack with nested ASN.1 encoding. IEC 60870-5-104 is a compact binary telemetry protocol carried directly over TCP with fixed-offset fields.

Across the three artifacts only the inputs differ: the captured traffic and the deployment configuration are per device, and the protocol description is per protocol. No runtime code was added for the second protocol. What remains genuinely protocol-specific is the transport module where the protocol has one, the rule implementations for the values a protocol computes, and a scheduler for devices that transmit unprompted. Table 1 reports what the classifier assigned in each case.

The first two devices, IEDs A and B, are bay control units of the same substation. Over IEC 61850 MMS they report switch positions and measured values to a station computer through the reporting model of IEC 61850-7-2 [11] and accept control commands from it. Their corpora hold 53,441 and 30,507 packets (3,871 and 2,551 application events over 69 and 65 sessions). As Table 1 shows, the classification is dominated by recorded content, because the object model, the service structure, and the quality descriptors are fixed for a given engineering configuration; the computed group holds echoed request identifiers, report sequence

Table 1 Field classification per device, counted over field occurrences in the constructed artifacts. Distinct field paths in parentheses. Computed includes the three state-rendered occurrences on each IED; the station computer has none.

Device	Total	Recorded	Computed	Generated
IED A (61850)	750	717 (147)	25 (12)	8 (4)
IED B (61850)	598	565 (146)	25 (12)	8 (4)
Station comp. (104)	358	200 (32)	147 (19)	11 (5)

numbers, entry timestamps, and the written-value list of a write response. Three field occurrences are rendered from tracked device state, since the switch position is determined by the control commands the device has received. Four field paths remain for the LLM, all carrying measured or reported process values.

The third device, the station computer of Figure 3, speaks IEC 60870-5-104. Its corpus holds 3,746 packets, taken from a mirroring port and deduplicated to 2,158. Sequence numbers, keepalives, and supervisory acknowledgements are per-connection transport state that no recorded template can carry; a transport module handles them outside the state machine, and the protocol description excludes the six transport message types, 455 of the corpus's 694 events, from the inferred state machine. The classification is the more interesting one for the argument of Section 3.2: of 358 field occurrences, 200 are recorded and 147 are computed, leaving 11 occurrences over 5 field paths for the LLM, because almost every value in a command confirmation is echoed from the command or recomputed. The value-population procedure found this distribution rather than our imposing it, and it supports sizing the LLM's share by the residual rather than by the protocol.

5. Evaluation

Section 5.2 answers RQ1 and Section 5.3 answers RQ2. RQ3 was answered by the case study of Section 4, which built all three artifacts with one code base, and Section 5.4 answers RQ4. Section 5.5 measures separately what the two redesigned mechanisms contribute, since that is what distinguishes this paper from the earlier report.

5.1 Setup

All captures come from the smart grid testbed authorized for this research, recorded at a mirroring port. The station computer corpus includes five adversary-in-the-middle captures in which

© 2026 Information Processing Society of Japan

the information-object address and the ON and OFF command values were manipulated on the wire. Construction and evaluation run on one fanless laptop (Apple M4, 32 GB unified memory, no discrete accelerator). The LLM of Section 3.1 is adapted with one LoRA adapter per device over 16 layers, trained for three passes at batch size 4 with a fixed seed, and decoded greedily; its held-out sets are grouped by prompt so that no conditioning appears in both training and test.

Fidelity is measured under five-fold cross-validation over captures: an artifact is built from the captures outside each fold and graded on those inside it, so every capture is answered by a build that never saw it. The fold builds are discarded; the deployable artifact is built from the whole corpus. Cross-validation rather than a single held-out split is necessary because the corpora are uneven: on IED A the four largest captures carry 80 % of all server frames, so a single split either grades too little traffic or grades an artifact built without most of the evidence. Folds are assigned by content hash, with the captures that arm the device's reporting spread across folds.

The comparison is per response packet: the emulator receives the captured client traffic, and its emitted response is compared with the real device's. A response counts as correct when it is byte-identical to the recorded one, or when every differing byte falls inside a field whose classification declares it exempt in advance: timestamps, sequence numbers, frame lengths, and opaque replayed identifiers are exempt, while recorded, echoed, state-rendered, and generated fields must match exactly. An unrecognized rule, a byte no classification covers, or a response of a different length all count as failures, so an unhandled case is never scored as a success. *Solicited* responses (sent because a client asked) and spontaneous reports (sent unprompted) are tallied separately. During fidelity measurement the LLM is disabled and generated fields fall back to their recorded template, so these rates measure the structure and the classification; the LLM is measured separately in Section 5.5.

5.2 Fidelity under identical traffic

Table 2 answers RQ1. The emulators reproduce 96.8 %, 93.7 %, and 98.9 % of the responses a client solicits, over 823, 510, and 88 graded responses. Every emitted response was structurally valid: the MMS responses decode against the standard service schema [15], and the station computer's responses re-frame as exactly one packet of a declared message type.

The two protocols reach that result differently, and the distinction is visible in the table. On the MMS devices, correctness and byte-identity coincide: every response counted correct was identical to the recorded bytes, and none was accepted on the strength of an exemption. On the station computer only 2 of 88 responses were byte-identical, while 85 differed exclusively inside the exempt timestamp that every packet of this protocol carries. Both outcomes are what the protocols imply. The MMS responses in this corpus carry no wall-clock field in their payload, so a correct answer is byte-identical; every station computer response carries one, so a correct answer cannot be.

The per-fold spread is the more informative number for a corpus this uneven. It is 91 to 99 % on IED A (standard deviation 3.2) and 96 to 100 % on the station computer (1.9), but 73 to 100 % on IED B (10.6). The weak fold on that device is the one holding out its association-heavy captures, where a fold that never observed a given association parameter set must answer from a neighboring template. This is the capture-coverage bound of Section 6 appearing as variance, and it is the expected cost of grading every capture with a build that did not observe it.

Table 2 Response fidelity against the real devices, pooled over five folds. A response is counted correct when it is byte-identical or differs only inside fields declared exempt. Spontaneous reports are tallied separately.

		Solicited			Reports
Device	*n*	Byte id.	Correct	Valid	Correct
IED A (61850)	823	96.8	96.8	100	1.6
IED B (61850)	510	93.7	93.7	100	2.0
Station comp. (104)	88	2.3	98.9	100	—

(Rates in %, *n* is a count. Report counts: 193 and 202; the station computer originates no reports in this corpus.)

Spontaneous reports are reproduced at 1.6 % and 2.0 %. The cause, discussed in Section 6, is that their content depends on the state of the physical plant, which the mechanism does not model.

A coverage figure belongs beside these rates. On IED A the emulator emitted a response for 1,016 of 2,938 captured server packets. The remainder is spontaneous report traffic, not unanswered requests: 23 of the 69 captures contain no client request at all, and the two largest captures are report bursts of hundreds of packets sent without being asked. The rates in Table 2 are therefore computed over the responses the emulator produced, so the reports it does not produce stay visible here rather than hidden in the denominator.

5.3 Reproducibility

RQ2 asks whether a measurement taken against the emulator can be repeated with the same result. We replay each session several times, each time from a fresh connection, and check whether the emulator emits identical bytes on every run. Two sources of harmless difference are removed first: the clock is fixed to a constant, since a timestamp field reads the current time and would differ on every run, and fields declared exempt, such as an identifier the renderer generates on its own, are masked before the comparison. For one capture per device we replayed each session five times, and the emitted bytes were identical across all five runs.

Identical bytes alone would prove little, because an emulator that ignored its input would also emit the same bytes every time. We therefore add a second check: the same session is replayed with the client packets in reverse order, and the output is required to change. It does, on every session with enough requests for the reordering to matter. Together the two checks show the reproducibility the assessment use case needs: the response depends on the protocol state and the request, not on the run.

5.4 Cost

RQ4 asks whether the mechanism is affordable on commodity hardware available at the site. Everything reported here ran on the fanless laptop of Section 5.1.

Construction from captures to artifact takes about 20 seconds per IED. Adapter training dominates and remains modest: 1,854,

© 2026 Information Processing Society of Japan

1,626, and 111 iterations over datasets of 2,475, 2,168, and 148 prompts, for roughly 40, 38, and 6 minutes. Each adapter occupies 11.8 MB, a size fixed by its configuration rather than the corpus.

On the two IEDs the request and response path carries no generative work, because the model renders only fields of the reports those devices send unprompted; on the station computer a few generated fields sit on that path. That path answers a read in 0.12 ms at the median, against the real IED's own median of 98.6 ms over 1,175 request and response pairs from the same captures, and association setup takes about 6 ms. A report with generative fields in the loop costs 1.2 to 1.9 s, dominated by roughly six model calls at 258 ms each; the lower end of that range falls within the real device's latency tail, whose slowest observed response is 1.35 s, and the upper end exceeds it. These latencies were measured on an earlier build of the MMS artifacts and not repeated for the station computer.

5.5 Effect of the two proposed mechanisms

FSM construction. Table 3 gives the state count at each step. Merging on message history alone collapses every device's state machine to a single state, the result the earlier report obtained: a one-state machine cannot distinguish a request seen before association from the same request seen after it. With lifecycle phases and the automatically identified merge constraints, the same corpora yield 17, 17, and 16 states. On the MMS devices the procedure confirms two of seven candidates, the service discriminator `mms.confirmedservicerequest` at precision 1.000 and the target object name at 0.979 and 0.938, and rejects the rest. The confirmed set is also a control surface: keeping only the service discriminator yields 4 states on both IEDs, separating transport, association, and the read-pending and write-pending data states, while adding the object name refines the data phase per target. On the station computer the procedure confirms four fields, including the cause of transmission and the information-object address.

Field classification. The classifier keeps only the residual on the LLM path, and Table 1 shows how small that residual is: of 750, 598, and 358 field occurrences per device, only 8, 8, and 11 reach the LLM. This offloading is what keeps the training cost of Section 5.4 low, since the adapter learns only those few field paths and not the values a rule or the recorded template already settles. The LLM is measured separately on the held-out prompts of Section 5.1: on the fields the classifier assigns to it, it reproduces the exact value in 77.1 % of 275 test cases on IED A and 81.7 % of 241 on IED B, with the value width correct in 94.2 % and 97.1 %. The station computer leaves only 16 test cases, which is too few to support an accuracy figure; its meaningful result is the classification distribution in Table 1.

Two observations connect these numbers to the classifier redesign. First, the procedure removed from the LLM path a field determined by earlier control commands; rendering it from tracked state is correct in 93.1 % of 613 occurrences under leave-one-out evaluation, against 40.6 % for the model that previously had to guess it. Second, the same procedure identified that a command confirmation on the station computer echoes `iec60870_asdu.ioa`, the address of the point being operated. Before that field was computed rather than replayed, the emulator confirmed whichever point its recorded template happened to name, and cross-validated fidelity on that device was 10.2 % pooled, with per-fold rates ranging from 0 to 100 % (standard deviation 41.5). Computing the field gives the 98.9 % of Table 2 and a per-fold standard deviation of 1.9. The size of that change, on a device whose model was untouched, is the clearest evidence that classification rather than model capacity was the binding constraint.

Table 3 State count per device and construction step. The prefix tree is built from paired transactions; the station computer's is counted after the transport message types are excluded.

Device	Prefix tree	Baseline	Proposed
IED A (61850)	3,331	1	17
IED B (61850)	2,076	1	17
Station comp. (104)	110	1	16

6. Limitations

The following limitations bound the claims of this paper.

Spontaneous reports. The fidelity of Section 5.2 holds for solicited responses; the report rates there are the visible symptom. Every emitted report is structurally valid, and the failures are almost entirely of one kind: the emulator emits a well-formed report of the wrong content, most often of a different length from the recorded one because it carries a different set of data points. The cause is structural rather than statistical. The content of a report depends on which measurements changed since the previous report. That is plant state, which the mechanism does not model and no amount of capture data supplies. The mechanism also does not reproduce the volume of report traffic, as the coverage figure in Section 5.2 shows. A tester whose scenario depends on the content of spontaneous reports, rather than on their presence and form, still needs the real device. Modeling the plant state that drives this traffic is the most substantial item of future work.

Scope of the behavioral claim. The mechanism reproduces the mapping from received bytes to sent bytes and nothing else. Firmware behavior, on-device code execution, and physical effects are out of reach by construction, and the reference system keeps real IEDs and station computers for the techniques that need them. In the vocabulary of the MITRE ATT&CK for ICS matrix [21], a reference system that combines this mechanism with the process simulators already on the testbed hosts 40 of 97 techniques through packet exchange alone and 21 more up to what is observable on the wire, roughly two thirds of the matrix, while the remainder, requiring firmware modification, on-device code execution, or physical damage, is reserved for real hardware. This is an analytical categorization, not a measurement; executing the matrix against a deployed reference system remains future work.

Capture coverage. Fidelity is bounded by what the captures contain. An unobserved input has no recorded behavior to reproduce; the state machine answers it from the closest matching transition, so the response is well formed but not guaranteed to match. The cross-validated design of Section 5.1 measures exactly this bound.

Single site. All three devices come from one site. The con-

© 2026 Information Processing Society of Japan

struction procedure contains nothing site-specific, and the second protocol required no change to the runtime code, but generality across sites is argued from the procedure rather than measured.

Protocol description authoring. The redesigns of Sections 3.2 and 3.3 enlarge the protocol description, which now declares the correlation field, the lifecycle phases, and the candidate fields the two confirmation procedures test. Writing it requires protocol knowledge, but the cost is paid once per protocol rather than once per device, and the candidate lists need only be plausible, since each candidate is confirmed against the corpus and dropped otherwise. The trade-off therefore pays off as soon as one protocol serves more than one device.

7. Conclusion and Future Work

We presented an emulation mechanism that reproduces the network-observable behavior of an installed ICS device from captured traffic alone, for use as a target in penetration testing of a reference system. Relative to the earlier report, field classification now decides how each field is reproduced from its observed values rather than from the name a dissector gives it, moving unpredictable fields off the LLM path and rule-computable ones into exact computation; and FSM construction now pairs transactions by correlation identifier, forbids merging across lifecycle phases, and confirms its merge constraints from the traces, raising the inferred state machine from a single state to a lifecycle-preserving one.

Measured under five-fold cross-validation, where every capture is graded by a build that never saw it, the emulators reproduce 96.8 %, 93.7 %, and 98.9 % of solicited responses on two IEC 61850 IEDs and one IEC 60870-5-104 station computer, with every emitted response structurally valid and every response identical across runs. The second protocol required captured traffic, a configuration, and a declarative protocol description, and no new runtime code.

Three directions follow. Modeling the plant state behind spontaneous reports is the limitation that most constrains coverage today; a report's content is fixed by that state, so this is a modeling problem, not a matter of more training data. A third protocol family with real-time multicast, such as IEC 61850 GOOSE, would test the description contract where its few-millisecond deadline leaves no room for the generative path. And applying the procedure at a second site would turn the single-site generality argument into a measurement.

Acknowledgments This article is based on results obtained from a project, JPNP24003, commissioned by the New Energy and Industrial Technology Development Organization (NEDO).

References

[1] Antonioli, D. and Tippenhauer, N. O.: MiniCPS: A Toolkit for Security Research on CPS Networks, *Proc. of ACM CPS-SPC'15*, New York, NY, USA, Association for Computing Machinery, p. 91–100 (online), DOI: 10.1145/2808705.2808715 (2015).

[2] Antunes, J., Neves, N. and Veríssimo, P.: Reverse Engineering of Protocols from Network Traces, *Proc. 18th Working Conference on Reverse Engineering (WCRE)*, pp. 169–178 (2011).

[3] Candell, R., Anand, D. M. and Stouffer, K.: A Cybersecurity Testbed for Industrial Control Systems, *Proc. of ISA PCS'14* (2014).

[4] Chamotra, S. and Barbhuiya, F. A.: Advancing Industrial Honeypots: FSM and LLM Integration for Realistic ICS Protocol Emulation, *IEEE Transa. on Industrial Informatics*, Vol. 22, No. 2, pp. 1038–1049 (online), DOI: 10.1109/TII.2025.3620426 (2026).

[5] Cherepanov, A.: WIN32/INDUSTROYER: A New Threat for Industrial Control Systems, ESET (online), available from ⟨https://web-assets.esetstatic.com/wls/2017/06/Win32_Industroyer.pdf⟩ (accessed 2026-08-06).

[6] Cintuglu, M. H., Mohammed, O. A., Akkaya, K. and Uluagac, A. S.: A Survey on Smart Grid Cyber-Physical System Testbeds, *IEEE Commu. Surveys & Tutorials*, Vol. 19, No. 1, pp. 446–464 (2017).

[7] Conti, M., Donadel, D. and Turrin, F.: A Survey on Industrial Control System Testbeds and Datasets for Security Research, *IEEE Commu. Surveys & Tutorials*, Vol. 23, No. 4, pp. 2248–2294 (online), DOI: 10.1109/COMST.2021.3094360 (2021).

[8] ESET Research: Industroyer2: Industroyer Reloaded, WeLiveSecurity, ESET (online), available from ⟨https://www.welivesecurity.com/2022/04/12/industroyer2-industroyer-reloaded/⟩ (accessed 2026-08-06).

[9] Green, B., Derbyshire, R., Knowles, W., Boorman, J., Ciholas, P., Prince, D. and Hutchison, D.: ICS Testbed Tetris: Practical Building Blocks Towards a Cyber Security Resource, *Proc. of USENIX CSET'20'* (2020).

[10] Hu, E., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S. and Chen, W.: LoRA: Low-Rank Adaptation of Large Language Models, *CoRR*, Vol. abs/2106.09685 (online), available from ⟨https://arxiv.org/abs/2106.09685⟩ (2021).

[11] IEC: IEC 61850-7-2 Ed. 2.1 — Communication Networks and Systems for Power Utility Automation, Part 7-2: Abstract Communication Service Interface (ACSI) (2020).

[12] IEC: IEC 61850-8-1 Ed. 2.1 — Communication Networks and Systems for Power Utility Automation, Part 8-1: Specific Communication Service Mapping (SCSM), Mappings to MMS (2020).

[13] IEC: IEC 62443-3-2 — Security for Industrial Automation and Control Systems, Part 3-2: Security Risk Assessment for System Design (2020).

[14] International Electrotechnical Commission: IEC 60870-5-104: Telecontrol equipment and systems — Part 5-104: Transmission protocols — Network access for IEC 60870-5-101 using standard transport profiles, Edition 2.0 (2006).

[15] ISO: ISO 9506-1:2003 and 9506-2:2003 — MMS Service Definition and Protocol Specification (2003).

[16] Kai, S., Kondo, T., Kasahara, K. and Murai, J.: Dynamic Penetration Testing to Evaluate Resilience Performance in OT/IT Hybrid Environments, *Proc. of SICE MSCS'26')* (2026).

[17] Kasahara, K., Kai, S. and Kondo, T.: OT/IO_EMU: An Emulation Mechanism for Efficient Penetration Testing of Industrial Control Systems without Real Hardware, *SIG Technical Reports on Computer Security*, Vol. 2026-CSEC-114, No. 104, pp. 1–8 (2026).

[18] Lang, K., Pearlmutter, B. and Price, R.: Results of the Abbadingo One DFA Learning Competition and a New Evidence-Driven State Merging Algorithm., *Proc. of ICGI'98*, pp. 1–12 (1998).

[19] López-Morales, E., Rubio-Medrano, C., Doupé, A., Shoshitaishvili, Y., Wang, R., Bao, T. and Ahn, G.: HoneyPLC: A Next-Generation Honeypot for Industrial Control Systems, *Proc. of ACM CCS'20*, pp. 279–291 (2020).

[20] Mathur, A. P. and Tippenhauer, N. O.: SWaT: A Water Treatment Testbed for Research and Training on ICS Security, *Proc. of CySWater'16*, pp. 31–36 (2016).

[21] MITRE: ATT&CK for ICS, The MITRE Corporation (online), available from ⟨https://attack.mitre.org/matrices/ics/⟩ (accessed 2026-06-01).

[22] Qwen Team: Qwen2.5: A Party of Foundation Models (2024).

[23] Roomi, M. M., Hussain, S. M. S., Chang, E., Nicol, D. M. and Mashima, D.: Auto-SGCR: Automated Generation of Smart Grid Cyber Range Using IEC 61850 Standard Models, *IEEE Open Journal of the Industrial Elec. Society*, Vol. 6, pp. 1575–1592 (2025).

[24] Stouffer, K. and Pease, M. and Tang, C.Y. and Zimmerman, T. and Pillitteri, V. and Lightman, S. and Hahn, A. and Saravia, S. and Sherule, A. and Thompson, M.: Guide to Operational Technology (OT) Security, Special Publication 800-82 Rev. 3, NIST (2023).

[25] Vasilatos, C., Mahboobeh, D. J., Lamri, H., Alam, M. and Maniatakos, M.: LLMPot: Dynamically Configured LLM-based Honeypot for Industrial Protocol and Physical Process Emulation, *Proc. IEEE EuroS&P'25*, pp. 963–979 (2025).

© 2026 Information Processing Society of Japan